

COMPUTER VISION PROJECT

Intelligent Face Recognition Attendance System

Student Project Guide • Version 1.0

Technology: Python • OpenCV • Face Recognition • MySQL • Streamlit • Git/GitHub

1. Project Overview

The Intelligent Face Recognition Attendance System is an end-to-end Computer Vision application. It captures a live camera stream, detects faces, recognizes registered students, and records attendance in MySQL. Streamlit provides the user interface and dashboard.

This project is intentionally job-oriented. Students are expected to understand not only the Computer Vision component, but also Python application structure, SQL/database design, UI development, testing, Git/GitHub, documentation, and deployment concepts.

2. Problem Statement

Manual attendance is time-consuming, error-prone, difficult to audit, and difficult to report at scale. The proposed application automates attendance for registered students using face recognition while maintaining structured records in MySQL.

3. Objectives

- Register students and capture multiple face images.
- Detect faces from a live camera feed.
- Recognize registered students.
- Record attendance automatically with date and time.
- Prevent duplicate attendance for the same student on the same day.
- Store student and attendance data in MySQL.
- Provide a Streamlit dashboard and reports.
- Practice modular Python application development.
- Use Git/GitHub for version control and portfolio development.

4. Technology Stack

Layer	Technology
Programming	Python
Computer Vision	OpenCV
Face Recognition	Face embeddings / suitable recognition library
Database	MySQL
UI / Dashboard	Streamlit
Data Processing	NumPy, Pandas
Database Connectivity	mysql-connector-python
Version Control	Git & GitHub

5. Functional Modules

Dashboard

Show total students, today's present/absent counts, attendance percentage, recent attendance, and useful charts.

Student Registration

Collect student information and capture/prepare face images and recognition data.

Mark Attendance

Open the camera, detect and recognize faces, validate identity, and record attendance.

Student Management

Search, view, update, deactivate, and manage student records.

Attendance Reports

Filter by student/date/date range and export attendance data.

Admin

Provide authentication and protected administrative functionality.

6. System Workflow

```
Student Registration
    ↓
Capture Face Images
    ↓
Face Processing / Encoding
    ↓
Store Student + Face Metadata
    ↓
Live Camera
    ↓
Face Detection
    ↓
Face Recognition
    ↓
Student Identified
    ↓
Check Existing Attendance
    ↓
Mark Attendance
    ↓
Store in MySQL
    ↓
Display Result in Streamlit
```

7. Project Structure

```
computer-vision-attendance/
|
├── app.py
├── pages/
|   ├── dashboard.py
|   ├── registration.py
|   └── attendance.py
```

```

|   |— reports.py
|   |— students.py
|   |— admin.py
|
|— src/
|   |— face_detection.py
|   |— face_recognition.py
|   |— database.py
|   |— attendance.py
|   |— authentication.py
|   |— utilities.py
|
|— dataset/
|— encodings/
|
|— database/
|   |— database.sql
|
|— .env
|— .gitignore
|— requirements.txt
|— README.md

```

8. Database Design

MySQL is the system of record for application data. A recommended initial schema contains students, attendance, users, and face_encodings. Actual biometric storage should be designed carefully; avoid placing raw face images directly in MySQL unless there is a clear requirement and security design.

```

CREATE DATABASE cv_attendance;
USE cv_attendance;

CREATE TABLE students (
    student_id VARCHAR(20) PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100),
    phone VARCHAR(20),
    course VARCHAR(100),
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

CREATE TABLE attendance (
    id INT AUTO_INCREMENT PRIMARY KEY,
    student_id VARCHAR(20) NOT NULL,
    attendance_date DATE NOT NULL,
    attendance_time TIME NOT NULL,
    status VARCHAR(20) DEFAULT 'Present',
    FOREIGN KEY (student_id) REFERENCES students(student_id),
    UNIQUE(student_id, attendance_date)
);

CREATE TABLE users (
    id INT AUTO_INCREMENT PRIMARY KEY,
    username VARCHAR(50) UNIQUE NOT NULL,
    password_hash VARCHAR(255) NOT NULL,
    role VARCHAR(20) DEFAULT 'admin'
);

CREATE TABLE face_encodings (
    id INT AUTO_INCREMENT PRIMARY KEY,

```

```

        student_id VARCHAR(20) NOT NULL,
        encoding_reference VARCHAR(255),
        created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
        FOREIGN KEY (student_id) REFERENCES students(student_id)
    );

```

9. Environment Setup

1. Install Python 3.10+ (use the version supported by the selected face-recognition dependency).
2. Install MySQL Server and MySQL Workbench or another MySQL client.
3. Create a project folder using the structure in this guide.
4. Create and activate a Python virtual environment.
5. Install dependencies from requirements.txt.
6. Create the MySQL database using database/database.sql.
7. Create a local .env file from the provided environment template.
8. Run the Streamlit application locally.
9. Create a Git repository and commit the working project.

```

python -m venv .venv

# Windows
.venv\Scripts\activate

# macOS/Linux
source .venv/bin/activate

pip install -r requirements.txt

streamlit run app.py

```

10. requirements.txt

```

streamlit
opencv-python
numpy
pandas
mysql-connector-python
Pillow
face-recognition
python-dotenv
openpyxl
bcrypt

```

Dependency versions should be pinned after the team confirms a working environment. Some face-recognition installations may require additional native build dependencies depending on the operating system.

11. Environment Variables

```

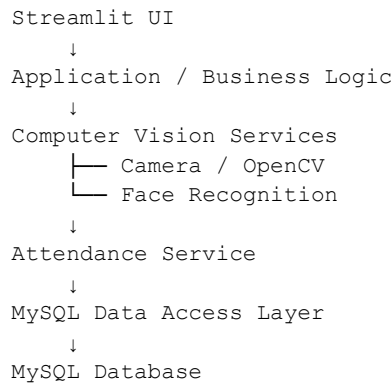
DB_HOST=localhost
DB_PORT=3306
DB_USER=root
DB_PASSWORD=your_password
DB_NAME=cv_attendance

APP_SECRET_KEY=change_this_value

```

Never commit the real .env file or database password to GitHub. Use .env.example for documentation and keep secrets in local environment configuration or a proper secret manager.

12. Application Architecture



13. Detailed Development Tasks

Phase 1 — Project Setup

10. Create folders and virtual environment.
11. Create requirements.txt, .gitignore, .env.example, and README.md.
12. Initialize Git repository.

Phase 2 — Database Layer

13. Create database.sql.
14. Implement database connection handling.
15. Implement student CRUD operations.
16. Implement attendance insert/query operations.

Phase 3 — Authentication

17. Create users table.
18. Hash passwords.
19. Implement login/session handling.
20. Protect administrative pages.

Phase 4 — Student Registration

21. Build Streamlit registration form.
22. Capture multiple face images.
23. Validate face presence and image quality.
24. Create/store face representations securely.

Phase 5 — Computer Vision

25. Implement camera capture.
26. Implement face detection.
27. Implement recognition.
28. Handle unknown and multiple faces.

Phase 6 — Attendance

29. Recognize a student.
30. Check whether attendance already exists.
31. Insert attendance record.
32. Show clear success/error feedback.

Phase 7 — Dashboard & Reports

33. Build dashboard metrics.
34. Add filters and date ranges.
35. Create attendance charts.
36. Add CSV/Excel export.

Phase 8 — Testing & Documentation

37. Write test cases.
38. Test edge cases.
39. Document installation and usage.
40. Prepare presentation, demo, and GitHub repository.

14. Streamlit Page Requirements

Dashboard

KPIs, today's attendance, recent records, and charts.

Registration

Student form, camera/image capture, validation, registration confirmation.

Attendance

Live camera, recognized identity, attendance status, duplicate prevention.

Reports

Date/student filters, table, summary metrics, CSV/Excel download.

Students

Search, view, update, deactivate student records.

Admin

Login and protected administration functions.

15. Computer Vision Requirements

- Detect faces from frames captured by the camera.
- Generate a consistent representation for registered faces.
- Compare live representations against registered representations.
- Use a documented recognition threshold and explain why it was selected.
- Handle unknown faces without creating attendance.
- Handle multiple faces safely.
- Provide user feedback when no face or low-quality input is detected.
- Test recognition under varied lighting, distance, pose, and camera conditions.

16. Attendance Business Rules

41. A student must be registered before attendance can be marked.
42. Only a sufficiently confident recognition result may be accepted.
43. Unknown faces must not be marked present.
44. A student can have at most one Present record per attendance date in the initial version.
45. Attendance must include student ID, date, time, and status.
46. Database constraints must back up application-level duplicate checks.
47. Failed database writes must be reported to the user and must not be presented as successful attendance.

17. Security & Privacy

- Never store plain-text passwords.
- Keep database credentials outside source code.
- Do not commit .env to Git.
- Restrict administrative functions to authenticated users.
- Limit access to biometric data and recognition files.
- Document data retention and deletion procedures.
- Use the project only with appropriate authorization and consent for biometric processing.
- Do not expose private student information in screenshots, GitHub repositories, or demo videos.

18. Testing Checklist

- Student registration succeeds with valid data.
- Invalid/missing student data is rejected.
- Camera can be opened and released correctly.
- Face detection works with a valid face.
- Unknown faces are not recognized as registered students.
- Multiple faces are handled without unsafe assumptions.
- Attendance is inserted correctly in MySQL.
- Duplicate attendance is prevented.
- Reports return correct records.
- CSV/Excel export contains expected data.
- Admin authentication blocks unauthorized access.
- Database failure produces a clear error.
- Application can be restarted without corrupting records.

19. Suggested Test Cases

Test ID	Scenario	Expected Result	Status
TC01	Register valid student	Student and recognition metadata saved	Pending
TC02	Register duplicate student ID	Operation rejected	Pending
TC03	Recognize registered student	Correct identity displayed	Pending
TC04	Unknown face	Unknown result; no attendance	Pending

TC05	Repeat same-day attendance	Duplicate prevented	Pending
TC06	Database unavailable	Clear error; no false success	Pending
TC07	Invalid login	Access denied	Pending
TC08	Attendance report filter	Correct records returned	Pending

20. Evaluation / Student Grading Rubric

Area	Weight	What is evaluated
Python & Code Quality	15%	Structure, readability, functions/classes, error handling
Computer Vision	25%	Detection, recognition flow, edge cases, evaluation
MySQL & SQL	15%	Schema, relationships, constraints, queries
Streamlit UI	15%	Usability, navigation, dashboard, forms
Testing	10%	Test coverage and edge cases
Git/GitHub	5%	Commits, branches/history, README
Documentation	5%	Technical/project documentation
Presentation & Viva	10%	Architecture, decisions, debugging, interview explanation

21. Interview Questions

- What problem does the project solve?
- Why did you choose Computer Vision?
- What is the difference between face detection and face recognition?
- What is a face embedding?
- How does the recognition threshold affect false positives and false negatives?
- How do you handle unknown faces?
- How do you handle multiple faces?
- Why did you choose MySQL?
- What are primary and foreign keys?
- Why is a UNIQUE constraint useful for attendance?
- How do you prevent SQL injection?
- Why should passwords be hashed?
- Why did you choose Streamlit?
- How would you scale the system for thousands of students?
- What happens if the database goes down during attendance?
- What challenges did you face and how did you solve them?
- How did you test recognition accuracy?

22. Resume Description

Intelligent Face Recognition Attendance System | Python, OpenCV, MySQL, Streamlit

Developed an end-to-end Computer Vision attendance application using Python and OpenCV to detect and recognize registered faces from a live camera stream. Integrated MySQL for persistent student and attendance data and built a Streamlit dashboard for registration, attendance monitoring, reporting, and administration. Implemented duplicate-attendance prevention, database relationships, authentication, error handling, and report generation.

23. Deliverables

- Complete source code.
- MySQL database.sql script.
- Streamlit application.
- Face registration and recognition modules.
- Attendance module.
- Reports module.
- README.md with setup and usage.
- GitHub repository.
- Test cases and results.
- Project presentation.
- Demo video.
- Final project report.

24. Git/GitHub Expectations

```
git init
git add .
git commit -m "Initial project setup"

git checkout -b feature/database
git checkout -b feature/registration
git checkout -b feature/attendance
git checkout -b feature/reports
```

Students should make meaningful commits. Do not upload passwords, API keys, private student images, or real biometric data to a public repository.

25. Definition of Done

- The application starts successfully from a clean environment.
- MySQL schema can be created from the SQL script.
- Admin authentication works.
- Students can be registered.
- Face data can be prepared for recognition.
- Registered students can be recognized from the camera.
- Attendance is stored in MySQL.
- Duplicate attendance is prevented.
- Dashboard and reports work.
- Application errors are handled gracefully.
- Tests have been executed and documented.
- README contains setup, configuration, usage, and troubleshooting.
- GitHub repository is clean and contains no secrets.

26. Important Implementation Note

The project should be implemented incrementally. Do not put all logic inside `app.py`. Keep database access, authentication, Computer Vision, attendance business rules, and Streamlit pages separated into modules. This makes the project easier to test, explain in interviews, and extend later.

The face-recognition component should also be treated as a real engineering problem: students must document the chosen library/model, representation format, threshold, dataset assumptions, limitations, and evaluation results. Recognition accuracy should never be presented as guaranteed.

27. Suggested Final Demo

48. Start the Streamlit application.
49. Log in as an administrator.
50. Register a sample student and capture face data.
51. Open Mark Attendance.
52. Show the camera detecting and recognizing the student.
53. Show the attendance confirmation.
54. Attempt a second attendance and demonstrate duplicate prevention.
55. Show the record in MySQL.
56. Open the dashboard and reports.
57. Filter attendance by student/date.
58. Export a report.
59. Explain the architecture and database schema.
60. Demonstrate GitHub repository structure.
61. Answer technical and project interview questions.

28. Student Submission Checklist

- Source code completed and organized.
- MySQL database script tested.
- `.env.example` included; secrets removed.
- README completed.
- Git history contains meaningful commits.
- All major modules tested.
- Demo video recorded.
- Presentation prepared.
- Test cases documented.
- Resume description prepared.
- Final project report submitted.

29. Conclusion

This project is designed to take students beyond a basic Computer Vision notebook and into an end-to-end application. By completing the system, students practice Python, Computer Vision, SQL/MySQL, Streamlit, software architecture, testing, security, version control, documentation, and technical communication.

The expected outcome is a portfolio-ready project that students can explain confidently in interviews and use as evidence of practical engineering ability.